ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №4

Проектирование и разработка БД, разработка серверной части

СОДЕРЖАНИЕ

1. [Введение](введение)
2. [Анализ требований](анализ-требований)
3. [Проектирование базы данных](проектирование-базы-данных)
4. [Реализация базы данных](реализация-базы-данных)
5. [Разработка серверной части](разработка-серверной-части)
6. [API endpoints](api-endpoints)
7. [Заключение](заключение)

1. ВВЕДЕНИЕ

Цель работы: Создать ER-диаграмму для базы данных, разработать структуру базы данных, реализовать серверную логику для обработки данных и обеспечить взаимодействие с фронтендом.

Задачи:

- Проанализировать ТЗ и use-case диаграмму для определения требований к данным

- Спроектировать схему базы данных, учитывая бизнес-логику и связи между сущностями

- Разработать и развернуть базу данных

- Реализовать серверную часть для обработки запросов

- Обеспечить обработку ошибок и валидацию данных

2. АНАЛИЗ ТРЕБОВАНИЙ

2.1. Основные сущности системы

На основе анализа предметной области выделены следующие сущности:

1. Пользователи (Users) - студенты и преподаватели системы

2. Курсы (Courses) - учебные планы и программы

3. Уроки (Lessons) - отдельные занятия в рамках курсов

4. Файлы (Files) - учебные материалы и ресурсы

5. Активность пользователей (User Activity) - логирование действий

2.2. Бизнес-правила

- Один пользователь может создавать множество курсов

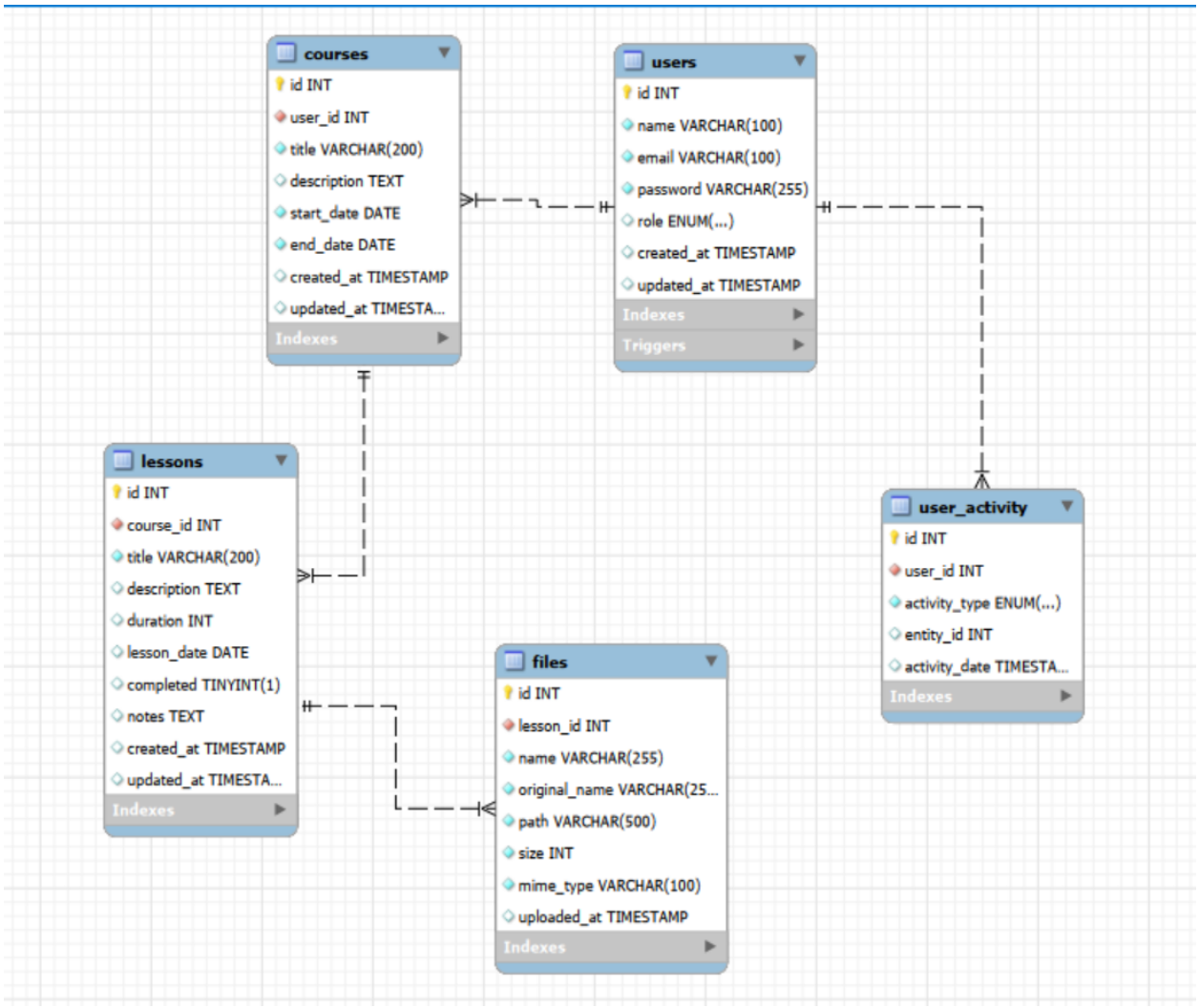
- Один курс содержит множество уроков

- К одному уроку может быть прикреплено множество файлов

- Система должна отслеживать активность пользователей

3. ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ

3.1. ER-диаграмма



3.2. Описание сущностей и атрибутов

Таблица: users

id	INT	Уникальный идентификатор	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(100)	Имя пользователя	NOT NULL
email	VARCHAR(100)	Электронная почта	UNIQUE, NOT NULL
password	VARCHAR(255)	Хэш пароля	NOT NULL
role	ENUM	Роль пользователя	ENUM('user', 'admin')
created_at	TIMESTAMP	Дата создания	DEFAULT CURRENT_TIMESTAMP
updated_at	TIMESTAMP	Дата обновления	ON UPDATE CURRENT_TIMESTAMP

Таблица: courses

id	INT	Уникальный идентификатор	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	Владелец курса	FOREIGN KEY → users.id
title	VARCHAR(200)	Название курса	NOT NULL
description	TEXT	Описание курса	NULL
start_date	DATE	Дата начала	NOT NULL
end_date	DATE	Дата окончания	NOT NULL
created_at	TIMESTAMP	Дата создания	DEFAULT CURRENT_TIMESTAMP
updated_at	TIMESTAMP	Дата обновления	ON UPDATE CURRENT_TIMESTAMP

Таблица: lessons

id	INT	Уникальный идентификатор	PRIMARY KEY, AUTO_INCREMENT
course_id	INT	Принадлежность к курсу	FOREIGN KEY → courses.id
title	VARCHAR(200)	Название урока	NOT NULL
description	TEXT	Описание урока	NULL
duration	INT	Продолжительность (мин)	DEFAULT 0
lesson_date	DATE	Дата проведения	NULL
completed	TINYINT(1)	Статус выполнения	DEFAULT FALSE
notes	TEXT	Заметки	NULL
created_at	TIMESTAMP	Дата создания	DEFAULT CURRENT_TIMESTAMP
updated_at	TIMESTAMP	Дата обновления	ON UPDATE CURRENT_TIMESTAMP

Таблица: files

id	INT	Уникальный идентификатор	PRIMARY KEY, AUTO_INCREMENT
lesson_id	INT	Принадлежность к уроку	FOREIGN KEY → lessons.id
name	VARCHAR(255)	Имя файла	NOT NULL
original_name	VARCHAR(255)	Оригинальное имя	NOT NULL
path	VARCHAR(500)	Путь к файлу	NOT NULL
size	INT	Размер (байты)	NOT NULL
mime_type	VARCHAR(100)	MIME-тип	NOT NULL
uploaded_at	TIMESTAMP	Дата загрузки	DEFAULT CURRENT_TIMESTAMP

Таблица: user_activity

id	INT	Уникальный идентификатор	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	Пользователь	FOREIGN KEY → users.id
activity_type	ENUM	Тип активности	NOT NULL
entity_id	INT	ID сущности	NULL
activity_date	TIMESTAMP	Дата активности	DEFAULT CURRENT_TIMESTAMP

3.3. Связи между таблицами

users → courses	1:n	Один пользователь создает много курсов
courses → lessons	1:n	Один курс содержит много уроков
lessons → files	1:n	К одному уроку прикреплено много файлов
users → user_activity	1:n	Один пользователь имеет много записей активности

4. РЕАЛИЗАЦИЯ БАЗЫ ДАННЫХ

4.1. SQL скрипт создания базы данных

```
CREATE DATABASE IF NOT EXISTS skilllvlup
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE skilllvlup;

-- Таблица пользователей
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  role ENUM('user', 'admin') DEFAULT 'user',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  INDEX idx_email (email),
  INDEX idx_role (role)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE courses (
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    user_id INT NOT NULL,
    title VARCHAR(200) NOT NULL,
    description TEXT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_user_id (user_id),
    INDEX idx_dates (start_date, end_date)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE lessons (
    id INT AUTO_INCREMENT PRIMARY KEY,
    course_id INT NOT NULL,
    title VARCHAR(200) NOT NULL,
    description TEXT NULL,
    duration INT DEFAULT 0 COMMENT 'Продолжительность в минутах',
    lesson_date DATE NULL,
    completed BOOLEAN DEFAULT FALSE,
    notes TEXT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE,
    INDEX idx_course_id (course_id),
    INDEX idx_completed (completed)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE files (
  id INT AUTO_INCREMENT PRIMARY KEY,
  lesson_id INT NOT NULL,
  name VARCHAR(255) NOT NULL,
  original_name VARCHAR(255) NOT NULL,
  path VARCHAR(500) NOT NULL,
  size INT NOT NULL COMMENT 'Размер в байтах',
  mime_type VARCHAR(100) NOT NULL,
  uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (lesson_id) REFERENCES lessons(id) ON DELETE CASCADE,
  INDEX idx_lesson_id (lesson_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE user_activity (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  activity_type ENUM('login', 'course_created', 'lesson_completed', 'file_uploaded') NOT
NULL,
  entity_id INT NULL COMMENT 'ID курса/урока/файла',
  activity_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  INDEX idx_user_id (user_id),
  INDEX idx_activity_date (activity_date),
  INDEX idx_activity_type (activity_type)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
-- Триггер для автоматического обновления updated_at
DELIMITER $$
CREATE TRIGGER before_user_update
BEFORE UPDATE ON users
FOR EACH ROW
BEGIN
   SET NEW.updated_at = CURRENT_TIMESTAMP;
END$$
DELIMITER ;
```

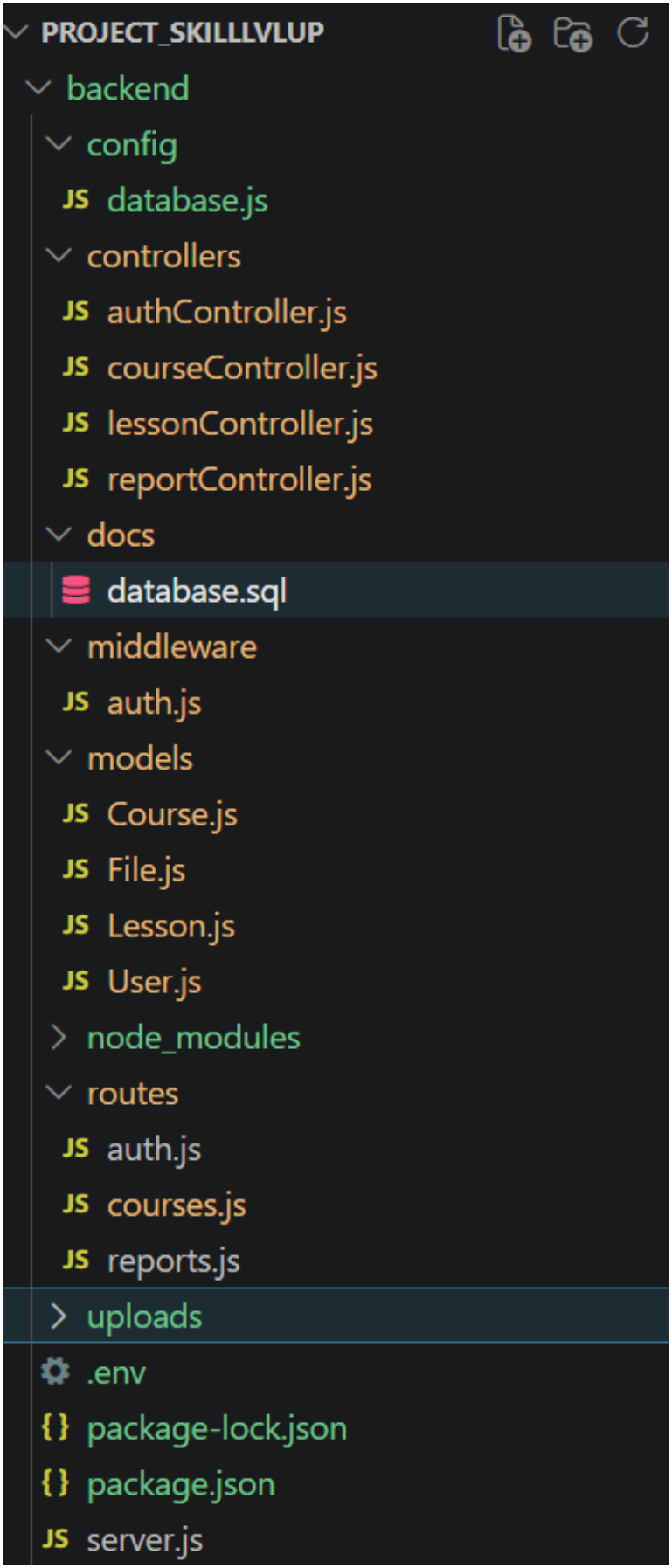
4.2. Особенности реализации

Индексы:

- Первичные ключи на всех таблицах
- Внешние ключи с каскадным удалением
- Индексы на часто используемых полях (email, user_id, course_id)
- Составные индексы для оптимизации запросов

5. РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ

5.1. Архитектура приложения



5.2. Технологический стек

Runtime	Node.js
Framework	Express
Database	MySQL
ORM	mysql2
Authentication	JWT (jsonwebtoken)
File Upload	multer
Security	bcrypt, cors

5.3. Основные модули

Конфигурация подключения к базе данных (config/database.js)

```
const mysql = require('mysql2/promise');
require('dotenv').config();

const pool = mysql.createPool({
    host: process.env.DB_HOST || 'localhost',
    user: process.env.DB_USER || 'root',
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME,
    port: process.env.DB_PORT || 3306,
    waitForConnections: true,
    connectionLimit: 10,
    queueLimit: 0,
    enableKeepAlive: true,
    keepAliveInitialDelay: 0
});

async function testConnection() {
    try {
        const connection = await pool.getConnection();
        console.log('База данных подключена успешно');
        connection.release();
        return true;
    } catch (error) {
        console.error('Ошибка подключения к БД:', error.message);
        return false;
    }
}

// Выполнение запроса
async function query(sql, params) {
    const [rows, fields] = await pool.execute(sql, params);
    return [rows, fields];
}

async function query(sql, params) {
    const result = await pool.execute(sql, params);
```

```
    console.log('Query result type:', Array.isArray(result) ? 'array' : typeof
result);
    console.log('Query result length:', Array.isArray(result) ? result.length :
'N/A');

    if (Array.isArray(result) && result.length === 2) {
        const [rows, fields] = result;
        console.log('Rows:', rows);
        return rows;
    }

    return result;
}

module.exports = { pool, testConnection, query };
```

Модель пользователя (models/User.js)

```
const { query } = require('../config/database');
const bcrypt = require('bcryptjs');

class User {
    static async create(name, email, password, role = 'user') {
        const hashedPassword = await bcrypt.hash(password, 10);
        const sql = `
            INSERT INTO users (name, email, password, role)
            VALUES (?, ?, ?, ?)
        `;
        const result = await query(sql, [name, email, hashedPassword, role]);
        return {
            id: result.insertId,
            name,
            email,
            role
        };
    }

    static async findByEmail(email) {
        const sql = 'SELECT * FROM users WHERE email = ?';
        const users = await query(sql, [email]);
        return Array.isArray(users) ? users[0] : null;
    }

    static async findById(id) {
        const sql = 'SELECT id, name, email, role, created_at FROM users WHERE id
= ?';
        const users = await query(sql, [id]);
        return users[0] || null;
    }
```

```
    static async verifyPassword(plainPassword, hashedPassword) {
        return await bcrypt.compare(plainPassword, hashedPassword);
    }
    static async update(id, name, email) {
        const sql = `
            UPDATE users SET name = ?, email = ? WHERE id = ?
        `;
        await query(sql, [name, email, id]);
        return this.findById(id);
    }

    static async getStats(userId) {
        const sql = `
            SELECT
                COUNT(DISTINCT c.id) as total_courses,
                COUNT(DISTINCT CASE WHEN c.end_date >= CURDATE() THEN c.id END)
as active_courses,
                COUNT(DISTINCT CASE WHEN l.completed = TRUE THEN l.id END) as
completed_lessons
            FROM users u
            LEFT JOIN courses c ON u.id = c.user_id
            LEFT JOIN lessons l ON c.id = l.course_id
            WHERE u.id = ?
        `;
        const stats = await query(sql, [userId]);
        return stats[0];
    }
}

module.exports = User;
```

Middleware авторизации (middleware/auth.js)

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

async function authenticateToken(req, res, next) {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.startsWith('Bearer ')
        ? authHeader.split(' ')[1]
        : authHeader;

    if (!token) {
        console.error('Токен не предоставлен');
        return res.status(401).json({ error: 'Токен не предоставлен' });
    }

    try {
        console.log('Проверка токена:', token.substring(0, 20) + '...');

        const decoded = jwt.verify(token, process.env.JWT_SECRET);
        console.log('Токен декодирован:', decoded);

        const user = await User.findById(decoded.userId);

        if (!user) {
            console.error('Пользователь не найден:', decoded.userId);
            return res.status(401).json({ error: 'Пользователь не найден' });
        }

        req.user = user;
        next();
    } catch (error) {
        console.error('Ошибка проверки токена:', error.message);

        if (error.name === 'TokenExpiredError') {
            return res.status(401).json({ error: 'Срок действия токена истек' });
        }

        return res.status(403).json({ error: 'Неверный токен' });
    }
}

function requireAdmin(req, res, next) {
    if (req.user.role !== 'admin') {
        return res.status(403).json({ error: 'Доступ запрещен' });
    }
    next();
}

module.exports = { authenticateToken, requireAdmin };
```

Сервер:

```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const path = require('path');
const fs = require('fs');

dotenv.config();

const { testConnection } = require('./config/database');
const authRoutes = require('./routes/auth');
const courseRoutes = require('./routes/courses');
const reportRoutes = require('./routes/reports');

const app = express();
const PORT = process.env.PORT || 3000;

const uploadsDir = path.join(__dirname, 'uploads');
if (!fs.existsSync(uploadsDir)) {
    fs.mkdirSync(uploadsDir, { recursive: true });
}

app.use(cors({
    origin: true,
    credentials: true,
    methods: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH', 'OPTIONS'],
    allowedHeaders: ['Content-Type', 'Authorization']
}));

app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.use('/uploads', express.static(uploadsDir));

app.get('/api/health', (req, res) => {
    res.json({
        status: 'OK',
        message: 'SkillLvlUp API работает',
        timestamp: new Date().toISOString(),
        version: '1.0.0'
    });
});

app.use('/api/auth', authRoutes);
app.use('/api/courses', courseRoutes);
app.use('/api/reports', reportRoutes);

app.use((req, res) => {
    res.status(404).json({ error: 'Маршрут не найден', path: req.path });
});
```

```
app.use((err, req, res, next) => {
    console.error('Ошибка:', err);
    if (err.code === 'ER_DUP_ENTRY') {
        return res.status(400).json({ error: 'Запись уже существует' });
    }
    if (err.code === 'ER_NO_REFERENCED_ROW') {
        return res.status(400).json({ error: 'Связанная запись не найдена' });
    }
    res.status(err.status || 500).json({ error: err.message || 'Внутренняя ошибка
сервера' });
});

async function startServer() {
    try {
        const dbConnected = await testConnection();
        if (!dbConnected) {
            console.error('Не удалось подключиться к базе данных');
            process.exit(1);
        }

        app.listen(PORT, () => {
            console.log('');
            console.log('====================================');
            console.log('SkillLvlUp Backend запущен');
            console.log('====================================');
            console.log(`Порт: ${PORT}`);
            console.log(`URL: http://localhost:${PORT}`);
            console.log('Uploads: ./uploads');
            console.log('====================================');
            console.log('');
        });
    } catch (error) {
        console.error('Ошибка запуска сервера:', error);
        process.exit(1);
    }
}

startServer();
```